How-To Library

Informatica

Creating a Data Processor Transformation for an XML Source

© Copyright Informatica LLC 2013, 2021. Informatica Corporation. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica Corporation. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

Use the Data Processor transformation to process an XML source with one format and produce an XML output in another format. The Data Processor transformation uses an XMap to transform an XML hierarchy into a different XML hierarchy. This article describes how to create and configure a Data Processor transformation with an XMap to transform an XML source.

Supported Versions

- Data Transformation 10.0 and higher

Table of Contents

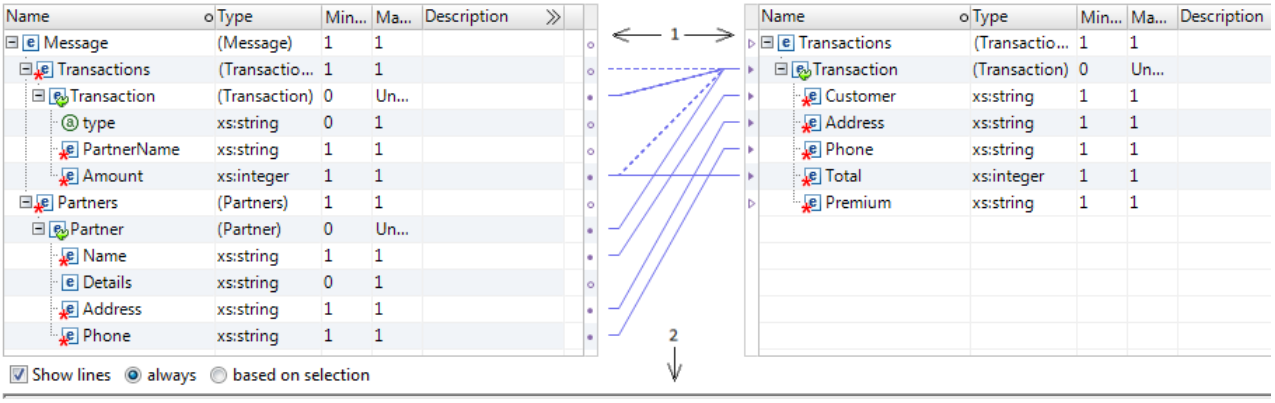
Transform an XML Source 2
Mapping Statement Types. 3
Editing Mapping Statement Fields. 4
Scenario. 5
XML Example Source. 5
XML Input Schema. 6
XML Output Schema. 7
Creating and Running a Mapping to Transform XML. 7
Step 1. Create an XMap in the Data Processor Transformation. 8
Step 2. Configure the XMap. 8
Step 3. Create and Run the Mapping 12
Import the Full Mapping. 13

Transform an XML Source

A mapping uses a Data Processor transformation to transform documents from one format to another. An XMap is a Data Processor transformation object that transforms an XML input source to another XML with a different hierarchy structure.

An XMap uses input and output schemas to define the expected hierarchy of the input and output XML. Use the XMap editor to define how input elements are mapped to output elements. An XMap can transform any input XML document whose elements match the input schema hierarchy into an output document with the hierarchy of the output schema.

The following figure shows the XMap editor:



	Name	Statement...	Input	Condition	Skip...	Default	On Fail	Output	Mode	Remark
1	Trans...	Repeating...	ns1:Transactions/ns...		☑		Skip Iterati...	ns2:Transaction[ns2:...	Match...	
2	tran...	Router			☐		Propagate		Match...	
3	buy	Option		@type="B"	☑		Propagate		Match...	
4	A	Map	if (exists(dp:output()...		☐		Propagate	ns2:Total	Match...	
5	sell	Option		@type="S"	☑		Propagate		Match...	
6	A	Map	if (exists(dp:output()...		☐		Propagate	ns2:Total	Match...	
7	Partne...	Repeating...	ns1:Partners/ns1:Par...		☐		Skip Iterati...	ns2:Transaction[ns2:...	Match...	
8	Na...	Map	ns1:Name		☑		Skip	ns2:Customer	Add	
9	Add...	Map	ns1:Address		☑		Skip	ns2:Address	Add	
10	Pho...	Map	ns1:Phone		☑		Skip	ns2:Phone	Add	

1. The XMap editor contains input and output XML schemas. Drag and drop between the schema elements to create mapping statements.
2. The XMap editor grid shows mapping statements. Use the grid to manage and edit the mapping statements.

An XMap uses mapping statements to define how to transform an input schema element to an output schema element. You can drag from a node in the input schema to a node in the output schema to create a link. When you create links, these are mapping statements.

The XMap editor shows the mapping statement in the grid. You can edit the mapping statements in the grid.

Mapping Statement Types

Mapping statement types define XMap mapping logic. Create a statement by dragging an input schema element to an output schema element, or adding a mapping statement to the grid.

You use the following types of mapping statements in this example:

Map

Maps a simple input element to a simple output element. A Map statement is the basic building block of the XMap.

Repeating Group

A group statement that the Data Processor transformation performs each time the input element appears in the input document. The Repeating Group contains Map statements which are iterated.

Router

Contains a group of Option statements, and selects only the Option statement whose condition criteria matches the input.

Option

One or more Option statements are nested under the Router statement. The Option statement defines a condition to map the input element to the output element.

Editing Mapping Statement Fields

Mapping statements contain fields that you can configure to customize the statement. Use the XPath editor to configure expressions. XPath is a query language used to select nodes in an XML document and perform computations.

You can configure the following mapping fields with XPath expressions:

Input

An XPath expression that evaluates to a one or more input elements or values. The mapping statement type identifies the input elements that the Data Processor transformation maps.

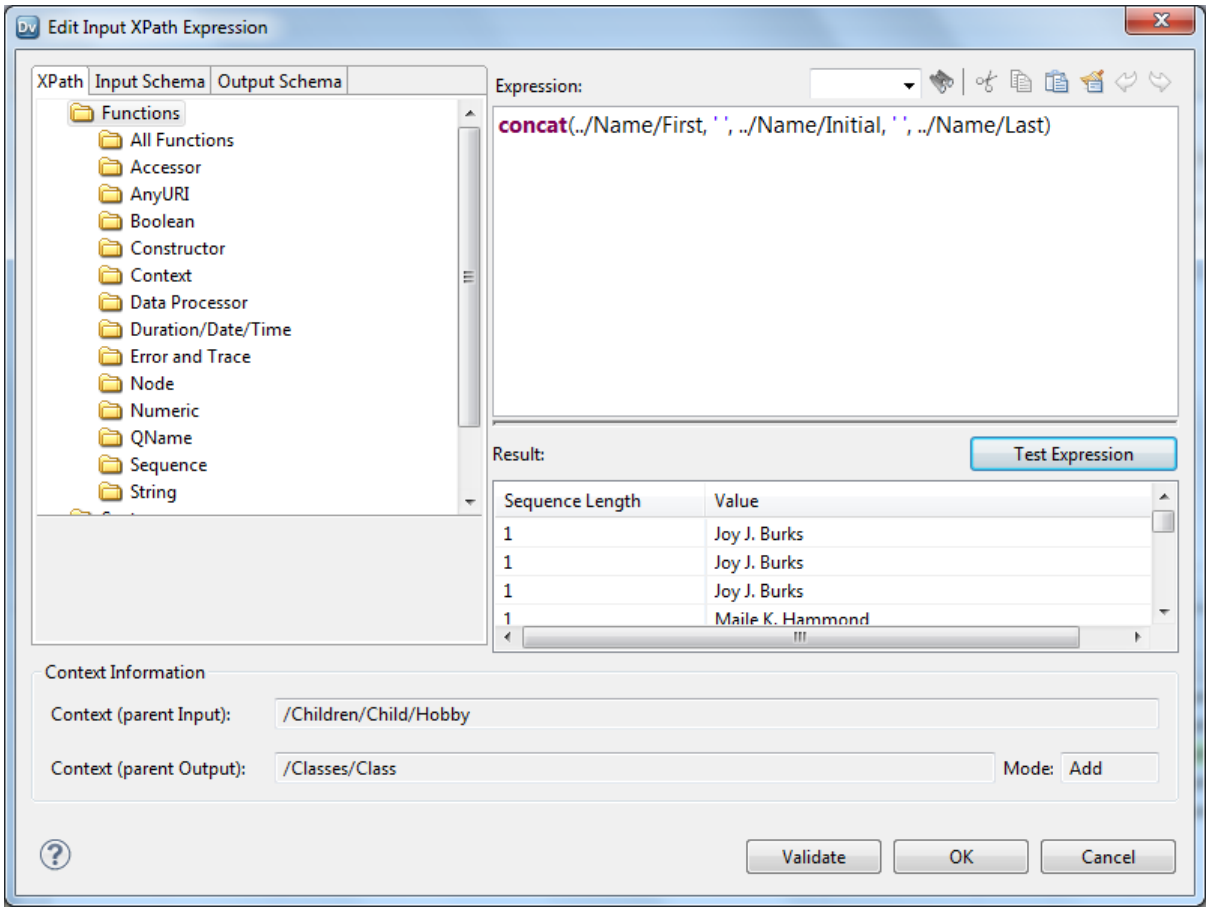
Condition

An XPath expression that defines a condition for mapping the element.

Output

An XPath expression that evaluates to one or more output elements or values. The mapping statement type identifies the output elements to which the Data Processor transformation maps.

XPath expressions identify specific elements in XML documents, or check for conditions in the data. Create expressions in the XPath Expression Editor. When you click the Open button to the right of the Input, Condition, or Output field, the Expression Editor appears:



Create expressions in the **Expression** panel.

The XPath Expression Editor has a **Navigation** panel with a function library that you can use to create XPath expressions. The functions are standard for the W3C XML Path Language. The function library also includes some functions that are specific to the Data Processor transformation.

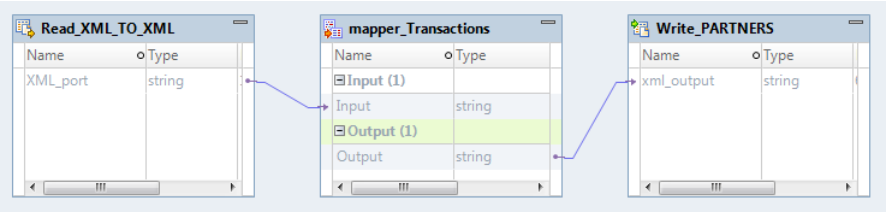
For further information about XPath expressions, refer to the *Data Transformation User Guide*, and to W3C XML Path Language.

Scenario

Investment Inc. uses an online tracking system to maintain customer transactions and contact details. The company can use their data to identify premium clients who most heavily use their services, as well as those who are less active clients.

They need to create a mapping that transforms daily logs with all buy and sell transactions into customer-specific data. The company online system stores transaction logs in XML format. The mapping needs to use a Data Processor transformation that inputs buy and sell transactions, sorts the data relevant to each customer, and outputs customer contact data and the total dollar amount of each customer's daily transactions..

The following figure shows the mapping in this example:



The mapping contains the following objects:

Read_XML_TO_XML

The source that contains the path to the file with transaction data and client data. Reads client and transaction data from an XML file.

mapper_Transactions

A Data Processor transformation that transforms an XML input hierarchy into an XML output hierarchy.

Write_PARTNERS

A target path to the file that stores the transformed data every time you run the mapping.

The mapping uses the Read_XML_TO_XML flat file to look up the transaction log. The mapping processes and transforms the data with the mapper_Transactions mapping. Then the mapping stores the output in the target path listed in the Write_Partners flat file.

XML Example Source

The following text shows sample data from the input XML document, Transactions-One.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<tr:Message xsi:schemaLocation="http://www.informatica.com/B2B/TransactionsInput
Transactions.xsd" xmlns:tr="http://www.informatica.com/B2B/TransactionsInput" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance">
    <tr:Transactions>
        <tr:Transaction type="S">
            <tr:PartnerName>Pregel Trading Co.</tr:PartnerName>
            <tr:Amount>1494</tr:Amount>
        </tr:Transaction>
        <tr:Transaction type="B">
```

```
            <tr:PartnerName>Touched Foundation</tr:PartnerName>
            <tr:Amount>7040</tr:Amount>
        </tr:Transaction>
        <tr:Transaction type="S">
            <tr:PartnerName>Touched Foundation</tr:PartnerName>
            <tr:Amount>2382</tr:Amount>
        </tr:Transaction>
...
...
    </tr:Transactions>
    <tr:Partners>
        <tr:Partner>
            <tr:Name>Quadrotect</tr:Name>
            <tr:Address>8173 Utrecht St., Tuscon AZ</tr:Address>
            <tr:Phone>9289283810</tr:Phone>
        </tr:Partner>
        <tr:Partner>
            <tr:Name>Xerth</tr:Name>
            <tr:Address>53453 Stanley St., Tuscon AZ</tr:Address>
            <tr:Phone>5207474710</tr:Phone>
        </tr:Partner>
...
    </tr:Partners>
</tr:Message>
```

The data can include multiple transaction and multiple partners. Each transaction is categorized as buy or sell. Each partner has contact information such as a name, address, and telephone number.

XML Input Schema

The XML Input schema for the XMap example is TransactionsSchemaIn.xsd. It has the following structure:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:ns1="http://
www.informatica.com/B2B/TransactionsInput" targetNamespace="http://www.informatica.com/B2B/
TransactionsInput" elementFormDefault="qualified" attributeFormDefault="unqualified">
    <xs:element name="Message">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="Transactions">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="Transaction" minOccurs="0" maxOccurs="unbounded">
                                <xs:complexType>
                                    <xs:sequence>
                                        <xs:element name="PartnerName" type="xs:string"/>
                                        <xs:element name="Amount" type="xs:short"/>
                                    </xs:sequence>
                                    <xs:attribute name="type" type="xs:string" use="optional"/>
                                </xs:complexType>
                            </xs:element>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
                <xs:element name="Partners">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="Partner" minOccurs="0" maxOccurs="unbounded">
                                <xs:complexType>
                                    <xs:sequence>
                                        <xs:element name="Name" type="xs:string"/>
                                        <xs:element name="Details" type="xs:string"
minOccurs="0"/>
                                        <xs:element name="Address" type="xs:string"/>
                                        <xs:element name="Phone" type="xs:string"/>
                                    </xs:sequence>
                                </xs:complexType>
                            </xs:element>
                        </xs:sequence>
                    </xs:complexType>
```

```
                </xs:element>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

The schema root is Message. Message has multiple-occurring Transactions and Partners.

Transactions contain Transaction elements with amount and type details. The type element determines whether the transaction is a buy or sell transaction.

Partners contain Partner elements with contact detail elements such as Name, Details, Address, and Phone.

XML Output Schema

The XML Output schema for the XMap example is TransactionsSchemaOut.xsd. It has the following structure:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:ns1="http://
www.informatica.com/B2B/TransactionsOutput" targetNamespace="http://www.informatica.com/B2B/
TransactionsOutput" elementFormDefault="qualified" attributeFormDefault="unqualified">
    <xs:element name="Transactions">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="Transaction" minOccurs="0" maxOccurs="unbounded">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="Customer" type="xs:string"/>
                            <xs:element name="Address" type="xs:string"/>
                            <xs:element name="Phone" type="xs:string"/>
                            <xs:element name="Total" type="xs:short"/>
                            <xs:element name="Premium" type="xs:string"/>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

The schema root is Transactions.

Transactions contain multiple-occurring Transaction elements. Within each Transaction element are Customer, Address, Phone, Total, and Premium elements.

Creating and Running a Mapping to Transform XML

To implement this scenario, you can import the mapping m_Transformation.xml that contains the Data Processor transformation, schemas, and data files already set up and ready to be used.

Alternatively, you can create a Data Processor transformation using the schemas and example source file from the Transformation_DP.zip, then add the transformation to a mapping. After complete the mapping, you can validate, save, and run the mapping.

1. Create an XMap in a Data Processor transformation.
2. Configure the XMap.
3. Add a Data Processor transformation to the mapping.
4. Run the mapping.

Step 1. Create an XMap in the Data Processor Transformation

Create a Data Processor transformation and then create an XMap for the transformation. When you create an XMap, you must have a schema that describes the input and the output XML documents. You select the element in the schema that is the root element for the input XML.

Before you begin, download the Transaction_DP.zip file from the following link:

https://kb.informatica.com/h2l/HowTo%20Library/1/0473-Transaction_DP.ZIP

Add the file to the `<INSTALL_DIR>/tomcat/bin/source` directory. Unzip the file to access the input schema file, output schema file, example source file, and sample mapping.

1. In the Developer Data Processor transformation **Objects** view, click **New.**
2. Select XMap and click **Next.**
3. Enter a name for the XMap.
4. The XMap component is the first component to process data in the transformation, so enable **Set as startup component**.

   Click **Next.**
5. To add a schema that defines the input, select **Add reference to a Schema Object**. Click **Create a new schema object** to import a new Schema object and browse for the **TransactionsSchemaIn.xsd** file in the `<INSTALL_DIR>/tomcat/bin/source` directory.
6. To add a sample XML file that you can use to test the XMap, browse for and select the **Transactions-One.xml** file in the `<INSTALL_DIR>/tomcat/bin/source` directory.

   You can change the sample XML file.
7. Choose the root for the input hierarchy.

   In the **Root Element Selection** dialog box, select the **Message** element in the schema as the root element for the XML. You can search for an element in the schema. You can use pattern searching. Enter `*<string>` to match any number of characters in the string. Enter `?<character>`to match a single character.
8. To add a schema that defines the output, select **Add reference to a Schema Object**. Click **Create a new schema object** to import a new Schema object and browse for the **TransactionsSchemaOut.xsd** file in the `<INSTALL_DIR>/tomcat/bin/source` directory.
9. Choose the root for the output hierarchy.

   In the **Root Element Selection** dialog box, select the **Transactions** element in the schema as the root element for the XML.
10. Click **Finish**.

    The Developer tool creates a view for each XMap that you create. Click the view to configure the XMap.

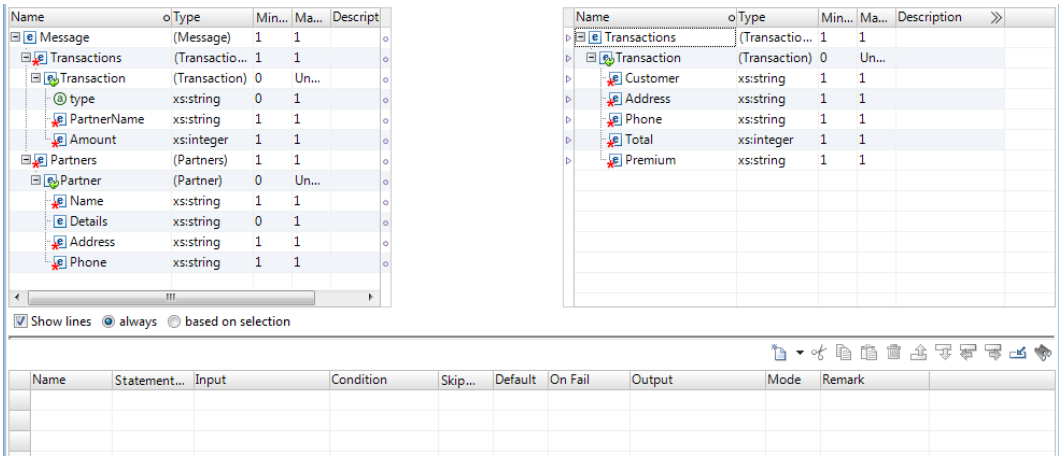
Step 2. Configure the XMap

Configure a Data Processor transformation XMap in the XMap editor. Create mapping statements by dragging nodes from the input schema to the output schema and define the statements in the mapping statement grid.

You want to map the client data as partner data. You want to compute how many transactions each client has performed and store this data.

1. To open the XMap editor, click the XMap object.

   The XMap editor displays the input schema to the left and the output schema to the right, as in the following figure:



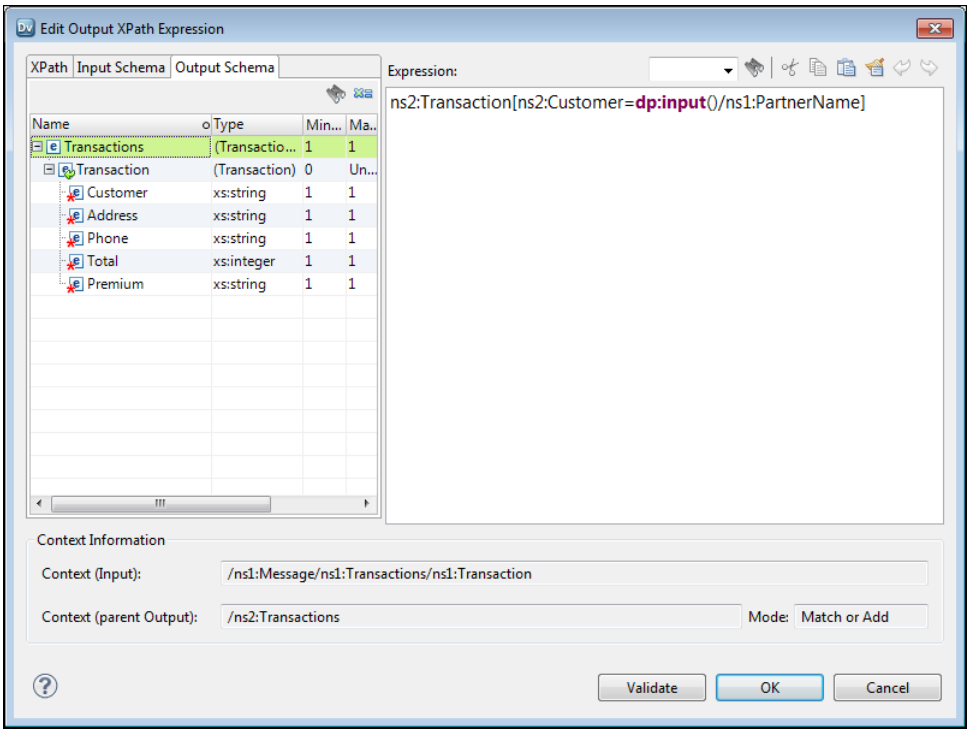
2. To map the total number of transactions per customer from the input XML to the output XML, complete the following steps:

   a. Drag the mouse from the **Transaction** input schema node to the **Transaction** output schema node.

      Both schema nodes are repeating nodes, so the XMap editor creates a Repeating Group statement in the grid. Statements under the Repeating Group statement are iterated for each instance of input data.

      To find the total transactions for each customer, search for transactions for the same customer using the PartnerName element with the dp:input() function. Use the XPath editor to enter the following expression in the **Output** field: `ns2:Transaction[ns2:Customer=dp:input()/ns1:PartnerName]`

      To open the XPath editor, click the arrow to the right of the **Output** field. The XPath editor displays the schema to the left and the expression to the right, as in the following figure:



   b. To create a Router statement that finds buy and sell transaction data, in the **Name** column, right-click and select **New** > **Router**. Name the statement **transaction type**.

      To ensure that the Router is part of the Repeating Group, right-click and select **Demote**.

   c. To create an Option statement that finds buy transaction data, in the **Name** column right click and select **New** > **Option**.

      Use the XPath editor to enter the following expression in the **Condition** field: `@type="B"`

   d. To add the amount of each buy transaction that the transformation finds in the input to the total transaction amount in the output, drag the mouse from the **Amount** input schema node to the **Total** output schema node.

      The XMap editor creates a Map statement in the grid.

      To ensure that the transformation only adds a transaction input amount if it exists, use the XPath editor to enter the following expression in the **Input** field: `if (exists(dp:output()/ns2:Total)) then`. Then use the XPath editor to enter the following expression in the **Output** field: `ns2:Total`

e. To create an Option statement that finds sell transaction data, in the **Name** column, right-click and select **New** > **Option**.

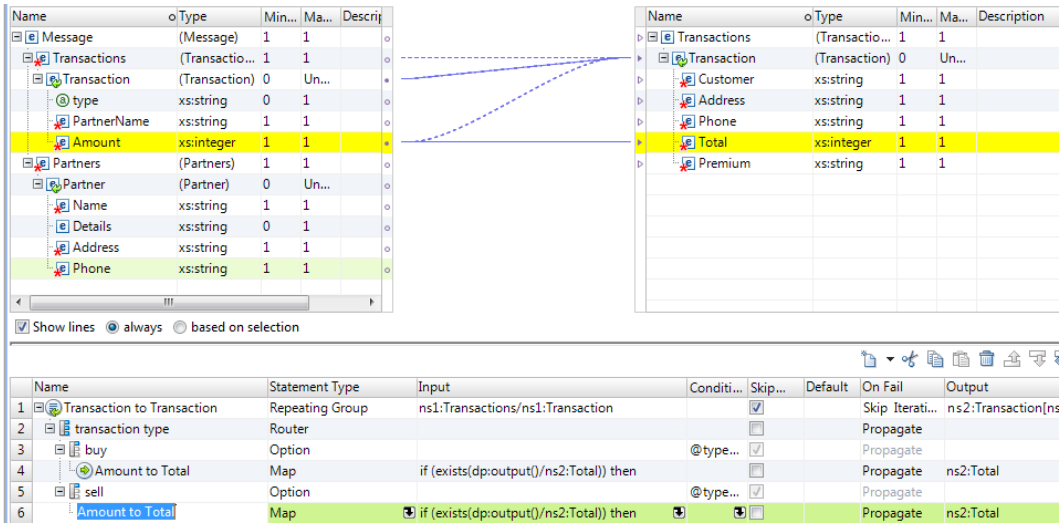
   Use the XPath editor to enter the following expression in the **Condition** field: `@type="S"`

f. To add a sell transaction to the total number of transactions in the output XML, drag the mouse from the **Amount** input schema node to the **Total** output schema node.

   The XMap editor creates a Map statement in the grid.

   Use the XPath editor to enter the following expression in the **Input** field: `if (exists(dp:output()/ns2:Total)) then`. Use the XPath editor to enter the following expression in the **Output** field: `ns2:Total`

The following figure shows how the XMap links the transaction data from the XML input to the XML output:



3. To map customer data from the XML input to the XML output, complete the following steps:

   a. Drag the mouse from the **Partner** input schema node to the **Transaction** output schema node.

      Both schema nodes are repeating nodes, so the XMap editor creates a Repeating Group statement in the grid. Statements under the Repeating Group statement are iterated for each instance of input data.

      Use the XPath editor to enter the following expression in the **Output** field: `ns2:Transaction[ns2:Customer=dp:input()/ns1:Name]`

   b. Drag the mouse from the **Name** input schema node to the **Customer** output schema node.

      The XMap editor creates a Map statement in the grid. This statement passes the name of the customer to the output XML.

   c. Drag the mouse from the **Address** input schema node to the **Address** output schema node.

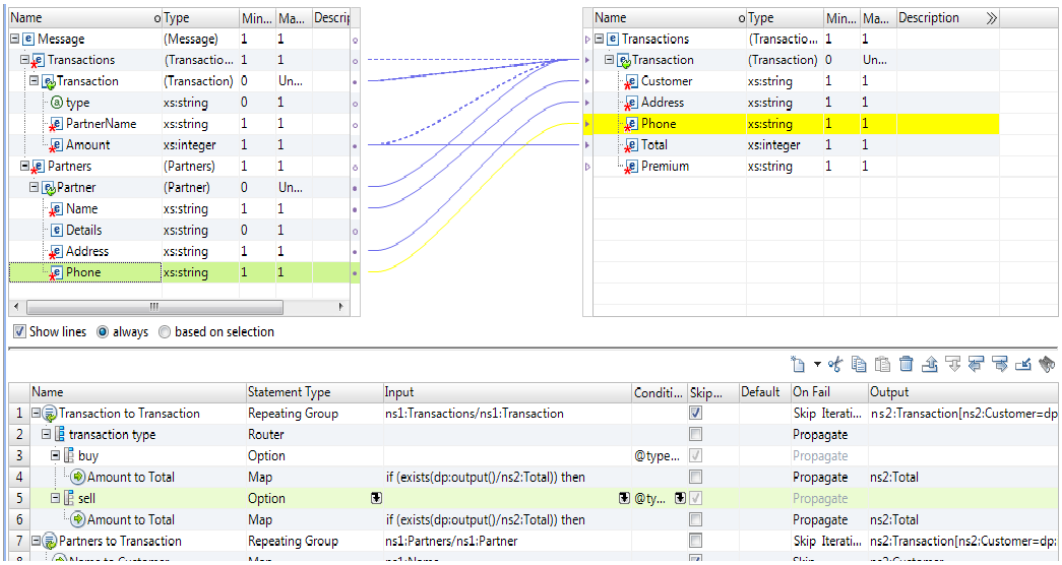
      The XMap editor creates a Map statement in the grid.

      This statement passes the customer address to the output XML.

   d. Drag the mouse from the **Phone** input schema node to the **Phone** output schema node.

      The XMap editor creates a Map statement in the grid. This statement passes the customer phone number to the output XML.

The following figure shows how the XMap links the customer data from the XML input to the XML output:

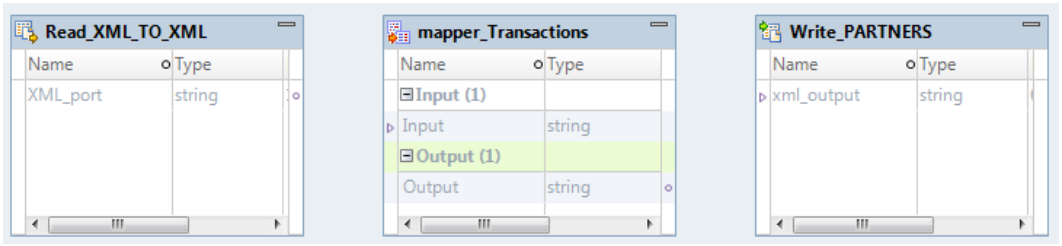


Step 3. Create and Run the Mapping

You can add the Data Processor transformation to a mapping and run the mapping.

1. In the **Object Explorer** view, create a mapping or select a mapping and select **Open Mapping**.
2. From the **Object Explorer** view, drag the Data Processor transformation into the editor.

   The following figure shows the mapping:

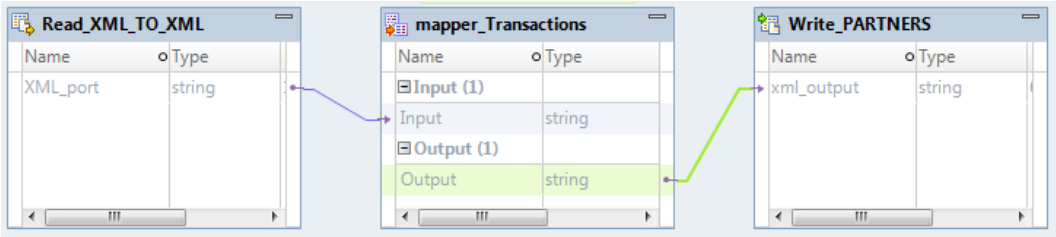


3. From the **Object Explorer** view, drag the Read_XML_TO_XML physical data object into the editor. Select **Read** to add the object as a source. The source appears in the editor. Drag the XML_port port in the source to the Input input port in the Data Processor transformation.

   The mapping reads input from the Transactions.xml when it runs.
4. From the **Object Explorer** view, drag Write_CLIENTS physical data object into the editor. Select **Write** to add the object as a target. The target appears in the editor. Drag the Output output port in the Data Processor transformation to the XML_output input port in the target.

   The mapping writes output to the Partners.xml when it runs.

The following figure shows the mapping:



5. Right-click in the editor, and select **Run Mapping**.

   Review the target flat file to see the mapping results.

Import the Full Mapping

If you want to check a prepared example mapping, you can import the full example mapping. The mapping contains the source flat file, transformation, and target flat file for the mapping.

1. In the **Object Explorer** view, select the folder where you want to create the mapping.
2. Right-click the folder and select **Import**.
3. Select **Informatica** > **Import Object Metadata File (Advanced)**.
4. Browse for the m_Transactions.xml file.
5. In the **Project Import dialog** box, select a folder and click **Add Content to Target**.

   For convenience, you can select a folder where you store practice examples.
6. Click **Next**, click **Next**, and click **Finish**.

   The Model Repository adds the m_Transactions mapping, the mapper_Transactions Data Processor transformation, and the input and output schemas. The m_Transactions mapping opens in the **Object Explorer** view.

Author

Rachel Aldam